



[Список тренингов](#) / [Промпт-инжиниринг](#) / [Промпт-инжиниринг 2.0](#) / [Модуль 8. Дообучение нейросетей \(/teach/control/stream/view/id/931478148\)](/teach/control/stream/view/id/931478148)

PE8.2 LLM Fine-tuning. Особенности дообучения

Предыдущий урок

[\(/pl/teach/control/lesson/view?id=340806117\)](/pl/teach/control/lesson/view?id=340806117)

Следующий урок

[\(/pl/teach/control/lesson/view?id=341125591\)](/pl/teach/control/lesson/view?id=341125591)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних заданий необходимо использовать исключительно тестовые

данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Результат дня

Сегодня мы обсудим fine-tuning (доработку) больших языковых моделей и рассмотрим особенности этого процесса.

Теория на сегодня

- Настройка LLM для дообучения, на какие параметры стоит смотреть.
- Оценка качества модели во время обучения, используемые метрики.
- Parameter-efficient fine-tuning (PEFT) — техники для экономии ресурсов при дообучении больших языковых моделей.

Время изучения материалов

▶ Время чтения и просмотра: ~ 1 час

🕒 Время выполнения задания: ~ 1 час

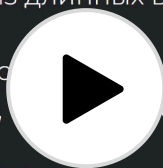
1. Особенности дообучения

Я — Иван Родин

- Постдок исследователь, Университет Катании
- Компьютерное зрение, анализ длинных видео, FPV, Action Anticipation
- Исследовательские проекты с Intel, Philips
- Публикации на CVPR, ICASSP, NAACL, CVIU

LinkedIn: <https://www.linkedin.com/in/mr-ivan-rodin/>

TG-Channel: https://t.me/aimazing_materials



zerocoder.ru

Эксперт — Иван Родин:

- Постдок исследователь, Университет Катании;
- компьютерное зрение, анализ длинных видео, FPV, Action Anticipation;
- исследовательские проекты с Intel, Philips;
- публикации на CVPR, ICASSP, NAACL, CVIU

LinkedIn: <https://www.linkedin.com/in/mr-ivan-rodin/>

(<https://www.linkedin.com/in/mr-ivan-rodin/>)

TG-Channel: https://t.me/aimazing_materials

(http://https://t.me/aimazing_materials)

Первый шаг в запуске fine-tuning больших языковых моделей — это выбор токенизатора. Важно использовать токенизатор, который соответствует исходному датасету. Токенизатор разбивает исходную строку символов на последовательность токенов, которые затем обрабатываются моделью.

Если заменить токенизатор, например, использовать более современный квантитокенизатор из модели LAMA вместо токенизатора модели BERT, исходная строка будет преобразована в другой набор токенов. Модель может воспринимать эти токены как бессмысленные, что снизит качество обучения.

я ем суп → [497, 1861, 2005] → я ем суп
я ем суп → [8, 171, 2, 102] → café @ www 6

Для переиспользования токенизатора можно воспользоваться библиотекой Transformers от Hugging Face. Инициализация токенизатора производится командой:

```
from transformers import AutoTokenizer

model_checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True)

tokenized_dataset = dataset.map(tokenize_function, batched=True) # Токени
tokenized_eval_dataset = eval_dataset.map(tokenize_function, batched=True)
```

Этот токенизатор можно использовать для разбивки обучающего и валидационного датасетов на токены, которые затем подаются на вход модели.

Для запуска обучения используется класс `transformers.Trainer`. Он принимает множество аргументов, но важнейшими являются:

- `model`: модель, которую обучаем;
- `args`: гиперпараметры обучения;
- `train_dataset/eval_dataset`: обучающая (обновляем веса модели)/валидационная (проверяем работу модели на новых данных) выборка;
- `tokenizer`: используемый токенизатор;
- `compute_metrics`: метрики качества, измеряемые при обучении

Чтобы узнать больше об аргументах этого класса, переходим по ссылке

https://huggingface.co/docs/transformers/v4.42.0/en/main_classes/trainer#trainer

(https://huggingface.co/docs/transformers/v4.42.0/en/main_classes/trainer#trainer)

2. Гипер-параметры обучения

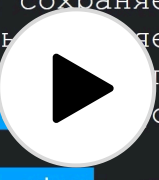
Гипер-параметры обучения

Класс `transformers.TrainingArguments()` : 112(!) аргументов

- `output_dir`: путь, куда сохраняем чекпойнты
- `learning_rate`: как сильно уменьшаем веса в процессе
- `num_train_epochs`: сколько эпох проходимся по датасету
- `fp16/half_precision_backend`: формат представления весов модели
- `per_device_train_batch_size`: сколько примеров одновременно обрабатываем на 1 устройстве (GPU)

https://huggingface.co/docs/transformers/v4.42.0/en/main_classes/trainer#transformers.TrainingArguments

zerocoder.ru



В процессе обучения или дообучения больших языковых моделей мы запускаем сложную вычислительную машину с множеством аргументов или гиперпараметров обучения. Эти гиперпараметры можно задать с помощью `transformers training arguments`, предоставляющих целых 112 различных параметров настройки. В этом руководстве мы рассмотрим основные параметры, которые особенно важны для успешного обучения моделей.

- `output_dir` — указывает директорию для сохранения чекпоинтов модели. Чекпоинты содержат конфигурацию и веса модели, что позволяет сохранять прогресс и использовать обновленную модель в будущем.

- `learning_rate` — это скорость обучения (`learning_rate`). Она определяет, насколько сильно изменяются веса модели в процессе обучения. Вы можете использовать стандартное значение или настроить его по своему усмотрению. Также существуют различные шедулеры для изменения скорости обучения в процессе тренировки.
- `num_train_epochs` указывает количество эпох обучения — сколько раз модель проходит по всему датасету. Обычно модели обучаются на протяжении нескольких десятков или сотен эпох. Важно учитывать ограничения по времени и ресурсам, особенно если обучение происходит в облаке, где оплата взимается за использованное время и вычислительные мощности.
- параметры, отвечающие за формат представления весов модели, позволяют использовать форматы `fp16` или `half_precision`. Это уменьшает занимаемую память и ускоряет процесс обучения.
- `per_device_train_batch_size` задает количество примеров, обрабатываемых одновременно на одном устройстве (обычно на одной видеокарте). Обычно используются степени двойки (1, 2, 4, 8, 16, 32 и т.д.), что ускоряет и стабилизирует процесс обучения. Если возникают ошибки, связанные с нехваткой памяти (`out of memory error`), нужно уменьшить значение `batch_size`, например, вдвое.

Если уменьшение `batch_size` не помогает и ошибки продолжаются, можно использовать меньшую модель или распределить нагрузку на несколько видеокарт. Это может потребовать отдельного урока по параллельной обработке данных на нескольких устройствах.

При fine-tuning или обучении моделей с нуля часто используется `SFTTrainer`. Этот класс предназначен для обучения моделей на примерах и особенно полезен для генерации новых текстов с помощью LLM.

Форматы данных для SFT Trainer

`SFT Trainer` принимает данные в двух форматах:

- **диалоговый формат** подходит для задач, аналогичных API чата GPT. Включает системный промпт, запрос от пользователя и ответ модели;

- **формат инструкций** — более простой формат, содержащий только промпт и ответ на него.

Выбор формата зависит от специфики решаемой задачи.

3. Метрики оценки качества моделей

Функции потерь

Функция потерь L — это мера расхождения между предсказанным моделью результатом и реальными данными.

Обучение модели — минимизация L для улучшения точности модели.

Проблема: интерпретируемость

Получить персональное видео



zerocoder.ru

В данном разделе мы обсудим метрики оценки качества моделей и рассмотрим, как их рассчитывать с помощью библиотеки Hugging Face. Основной целью обучения моделей является минимизация функции потерь, что позволяет улучшить точность предсказаний модели.

👉 Функция потерь — это мера расхождения между предсказаниями модели и реальными данными. Если предсказания модели полностью совпадают с реальными данными, значение функции потерь равно нулю. В противном случае функция потерь принимает положительное значение, и оптимизатор стремится минимизировать это значение, обновляя веса модели.

Основная проблема с функцией потерь заключается в её неинтерпретируемости. Например, в задаче классификации часто используется функция потерь, называемая бинарной кросс-энтропией. Если значение функции потерь на датасете равно 0.42, сложно сказать, много это или мало. Для одного датасета это может быть хорошим результатом, для другого — нет.

Для оценки качества работы модели важно использовать метрики, которые легко интерпретируются. Одной из таких метрик является точность (accuracy), особенно в задачах классификации.

Точность показывает, какой процент предсказаний модели является правильным. Например, если точность модели составляет 90%, это означает, что в 90% случаев модель делает правильные предсказания, а в 10% — ошибается. Точность не зависит от количества примеров в датасете или распределения классов, что делает её удобной для сравнения моделей на разных датасетах.

Во время обучения важно мониторить изменения метрик, чтобы оценить, насколько хорошо модель работает. В библиотеке Hugging Face для этого используются функции, которые позволяют рассчитывать и отслеживать метрики на каждой эпохе обучения.

Метрики для задач классификации

Когда речь заходит о метриках для задач классификации, существует несколько ключевых метрик, которые широко используются и являются основными. Самая простая метрика классификации — это **точность (accuracy)**, которая представляет собой процент правильных ответов модели.

Основные метрики для бинарной классификации:

- True Positive (Истинно положительные, TP) — это случаи, когда модель правильно предсказывает положительный класс (например, модель предсказала, что женщина беременна, и она действительно беременна);

- True Negative (Истинно отрицательные, TN) — это случаи, когда модель правильно предсказывает отрицательный класс (например, модель предсказала, что женщина не беременна, и она действительно не беременна);
- False Positive (Ложно положительные, FP) — это случаи, когда модель предсказывает положительный класс, но в реальности класс отрицательный (например, модель предсказала, что женщина беременна, но она не беременна);
- False Negative (Ложно отрицательные, FN) — это случаи, когда модель предсказывает отрицательный класс, но в реальности класс положительный (например, модель предсказала, что женщина не беременна, но она беременна).

В зависимости от задачи, может быть важно минимизировать либо ложные срабатывания (FP), либо ложные пропуски (FN). Для этого используются метрики Precision и Recall.

Точность (Precision): Доля правильных положительных результатов среди всех предсказанных положительных.

Полнота (Recall): Доля правильно идентифицированных положительных результатов из всех реальных положительных.

F1-Скор (F1-Score): Среднее между точностью и полнотой, оценивает баланс между ними.

Метрики для генерации текстов более сложны и разнообразны, так как задача оценки качества сгенерированного текста сложнее, чем классификация.

Метрика	Решаемая проблема	Описание
ROUGE	Суммаризация	Измеряет перекрытие N-грамм и LCS (наименьшей общей последовательности) между генерациями и эталонными текстами. Если в генерации есть последовательность из трёх слов, которые встречаются в эталонных текстах, то метрика будет равна 3.

Метрика	Решаемая проблема	Описание
BLEU	Машинный перевод	Измеряет точность N-грамм между генерацией и эталонным переводом
METEOR	Машинный перевод	Гармоническое среднее между точностью и полнотой униграмм, с штрафом за несоответствие длины
BERTScore	Схожесть текстов	Косинусное сходство между контекстуализированными векторами слов в кандидате и эталонных предложениях. Попарно сравниваются эмбединги сгенерированных и эталонных текстов.
Perplexity	Схожесть текстов	Измеряет, насколько хорошо модель предсказывает текст, оценивая степень неопределённости в выборе следующего слова.

Hugging Face предоставляет библиотеку `Evaluate` для расчета и мониторинга метрик. Ранее для этого использовался модуль в библиотеке `Dataset`, но теперь рекомендуется использовать `Evaluate`.

Документация библиотеки: <https://huggingface.co/evaluate-metric> (<https://huggingface.co/evaluate-metric>)

Пример использования библиотеки:

```
import evaluate
rouge = evaluate.load('rouge')
predictions = ["hello there", "general kenobi"]
references = [["hello", "there"], ["general kenobi", "general yoda"]]
results = rouge.compute(predictions=predictions, references=references)
print(results)
```

Метрики из библиотеки Evaluate можно адаптировать под конкретные задачи, а также создавать кастомные метрики для специфических нужд.

Можно создать собственную метрику, написав функцию, которая получает на вход предсказания модели и истинные значения, а затем вычисляет требуемую метрику.

При использовании ROUGE предсказания могут выглядеть так. Вычисляются несколько величин:

```
{'rouge1': 0.8333, 'rouge2': 0.5, 'rougeL': 0.8333, 'rougeLsum': 0.8333}
```

- rouge1 — пересечение униграмм (индивидуальных слов) между истинными значениями и предсказанными;
- rouge2 — пересечение биграмм (последовательностей из двух слов).

Мы можем найти кастомные метрики на HuggingFace или создать собственные метрики с помощью функции. При неумении кодить мы можем получить функцию от ChatGPT или другой нейросети.

```
def compute_average_length(predictions, references):
    total_length_pred = sum(len(pred.split()) for pred in predictions)
    total_length_ref = sum(len(ref.split()) for ref in references)
    avg_length_pred = total_length_pred / len(predictions)
    avg_length_ref = total_length_ref / len(references)
    return {
        "average_length_pred": avg_length_pred,
        "average_length_ref": avg_length_ref,
    }

# Example data
predictions = ["This is a test.", "Another test sentence."]
references = ["This is a test!", "Yet another sentence for testing."]

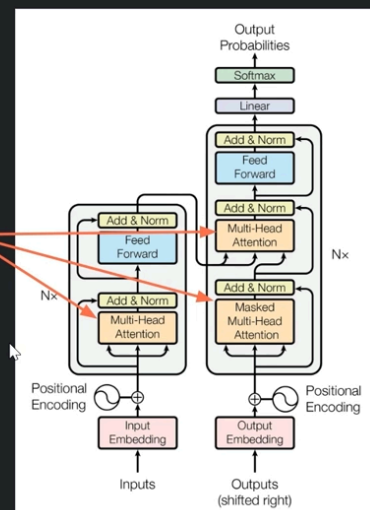
# Compute the metric
results = compute_average_length(predictions, references)
print(results)

{'average_length_pred': 3.5, 'average_length_ref': 4.5}
```

4. Parameter Efficient Fine-tuning (PEFT), LoRA

Low-Rank Adaptation (LoRA)

Наиболее “тяжелые” слои в LLM: **Self-attention**



Минусы Full fine-tuning (когда мы обновляем все параметры модели):

- обучение модели — ресурсоемкий процесс, требует в 5-6* раз больше ресурсов, чем инференс;
- если хотим тюнить модель для N разных задач, возникает проблема с хранением и запуском N моделей.

Чтобы решить проблему высоких затрат ресурсов, предложены подходы, которые заключаются в обучении частей модели, а не всей модели целиком.

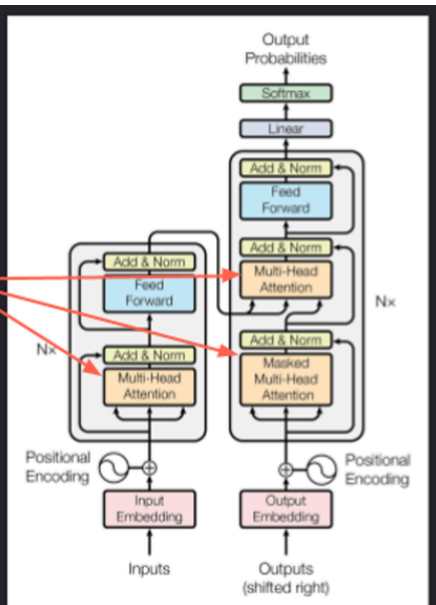
PEFT — это подход к обучению больших языковых моделей с минимальными затратами ресурсов.

- **Selective Methods** — обучаем только выбранные параметры модели. Какие-то слои замораживаем, какие-то обучаем;
- **Reparameterization** — адаптируем слои модели, чтобы они содержали меньше обучаемых параметров. Для этого заменяем ресурсоёмкие слои модели (*LoRA*);
- **Additive Methods** — добавляем новые обучаемые слои к модели, замораживая основную часть модели (*Proft Tuning, Soft Prompts*).

Рассмотрим Low-Rank Adaptation (LoRA)

👉 LoRA — это метод, который позволяет обучать меньшую часть параметров модели. Веса внутри блока внимания представлены в виде матрицы, и мы добавляем две матрицы разложения ранга R, обучая только веса этих матриц.

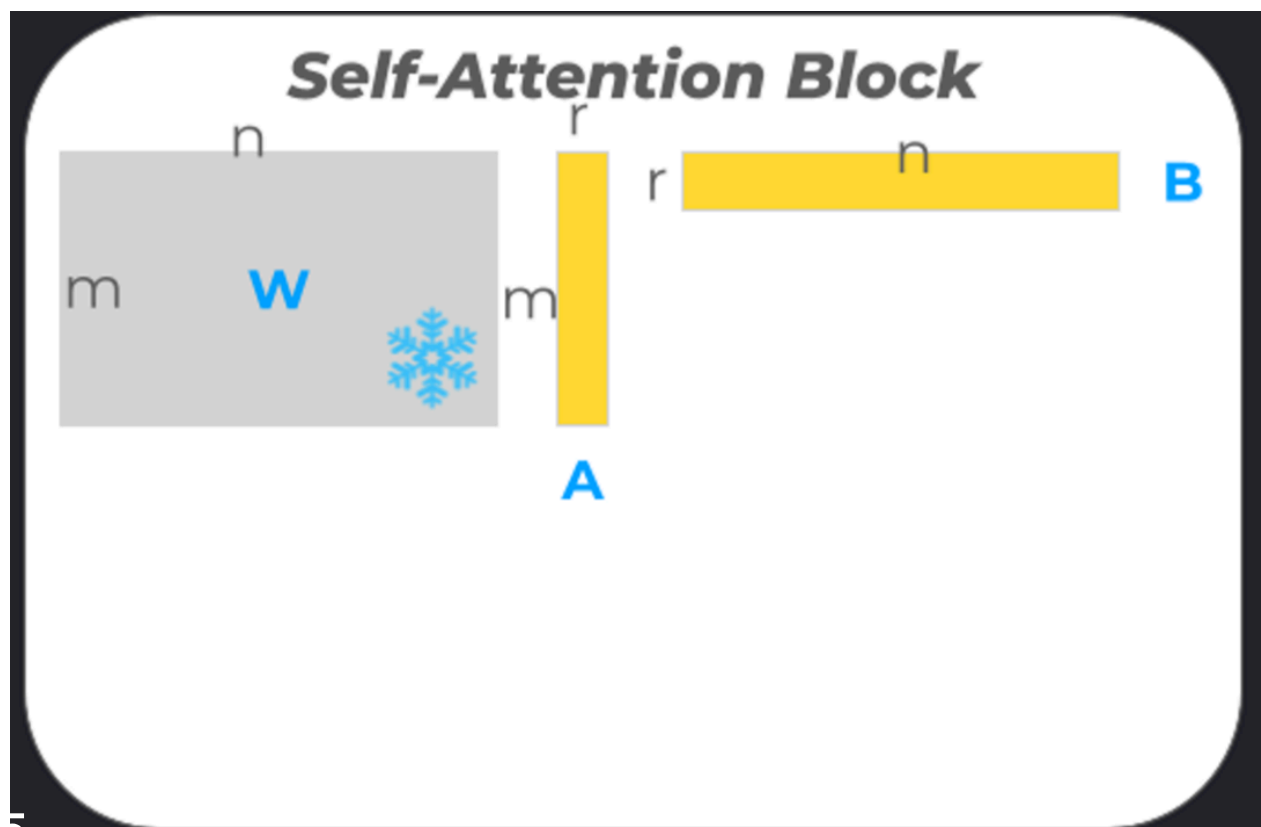
Наиболее “тяжелые” слои в LLM: **Self-attention**



Преимущества LoRA:

- экономия ресурсов: обучается значительно меньше параметров;
- высокое качество: несмотря на уменьшение количества обучаемых параметров, качество модели остается высоким.

1. Замораживаем веса в матрице, не обновляем их.
2. Добавляем две матрицы разложения ранга R . При перемножении эти матрицы дадут матрицу нужного размера.



3. Обучаем только веса матриц ранга R.
4. Перемножаем матрицы ранга R.
5. Добавляем новую матрицу к оригинальной матрице весов.

Этот подход работает, потому что:

- мы адаптируем слой с помощью двух матриц;
- мы обучаем некоторые веса;
- мы сохраняем изначальные данные и дополняем их перемноженной матрицей.

Пример экономии ресурсов:

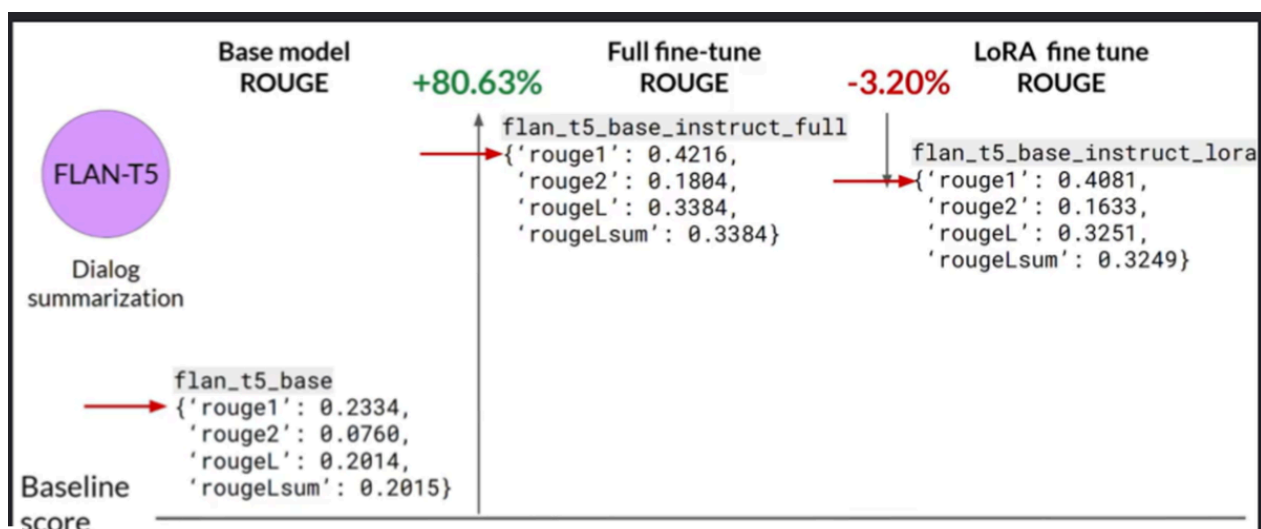
Файн-тюним базовый трансформер (Vasmani et al, 2017)

- Веса в self-attention имеют размерность $d \times k = 512 \times 64$
- $512 \times 64 = 32,768$

Используем LoRA с рангом $r=8$

- Матрица A: $512 \times 8 = 4096$ параметров
- Матрица B: $8 \times 64 = 512$ параметров
- В сумме: 4608 параметров, **на 86% меньше!**

То есть мы обучаем меньше параметров и улучшаем метрику — пусть и несколько ниже, чем при Full Fine-Tuning.



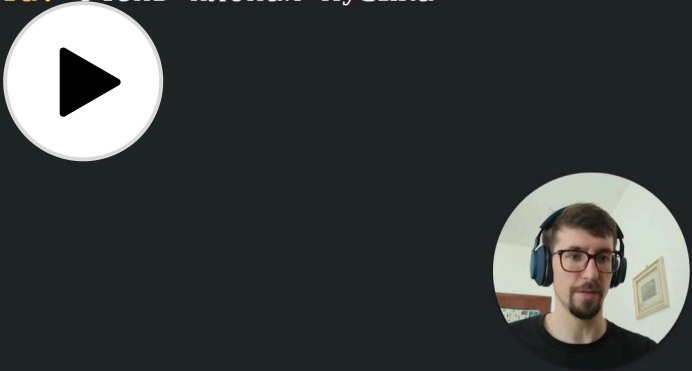
5. Prompt-Tuning

Prompt Engineering VS Prompt Tuning

Промпт-инжиниринг: составляем качественный промпт

Определи тональность текста: очень плохая музыка

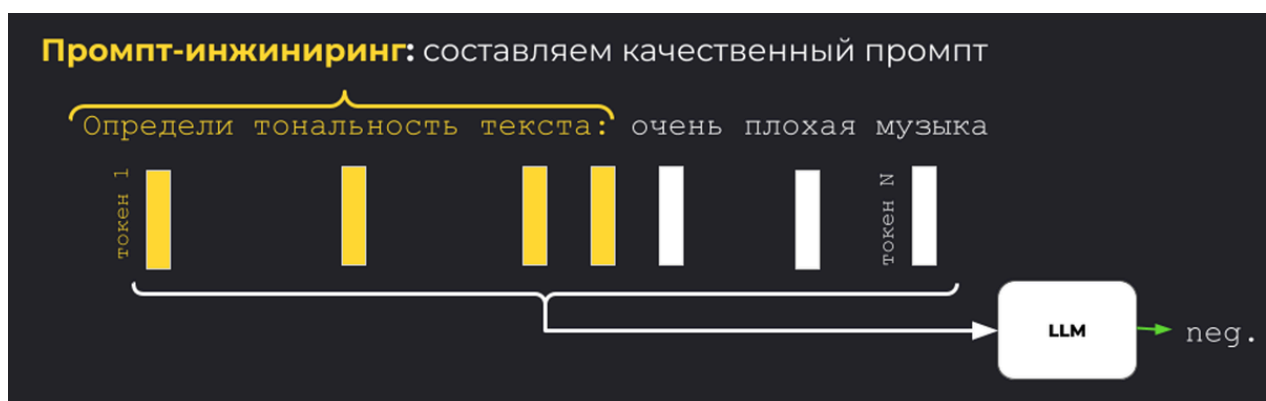
zerocoder.ru



Prompt Tuning — это подход, при котором исходные тексты дополняются обучаемыми имбедингами, создавая идеальный промпт на латентном уровне.

Не путаем с Prompt Engineering.

Для каждого токена из промпта и выдачи мы получаем эмбединги, которые объединяются и передаются в модель.

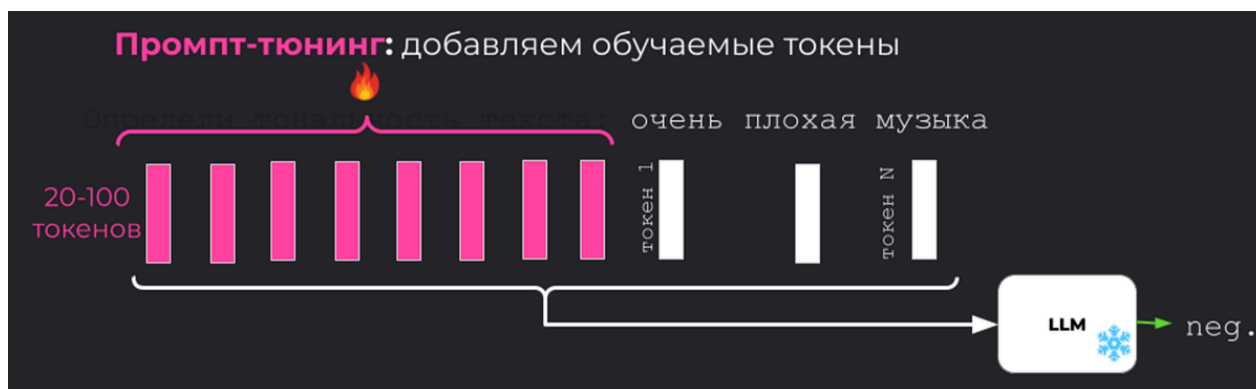


Ограничения пропт-инжиниринга:

- улучшение качества может быть незначительным — при решении новой задачи можно получить приемлемые результаты, но не идеальные;
- длинный, расписанный промпт съедает большую часть контекста, а контекст в больших языковых моделях ограниченный.

Промпт-тюнинг состоит в другом.

1. Токенизируем исходный запрос, получаем эмбединги.
2. Дополняем эмбединги обучаемыми эмбедингами. Это векторы, которые содержат изменяемые параметры.
3. Замораживаем языковую модель и обновляем только обучаемые токены.

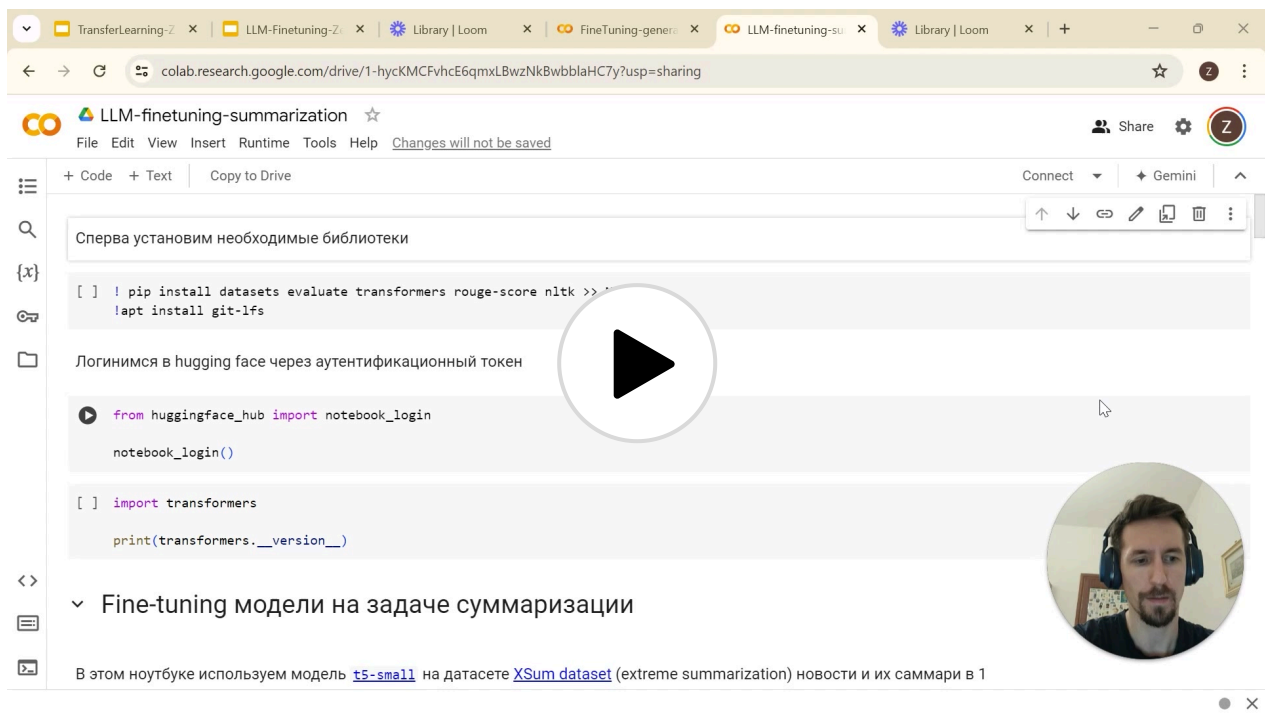


Так мы получим невербализованный идеальный промпт, который подходит для решения задачи, хотя и не может быть никак выражен.

Преимущества Prompt Tuning

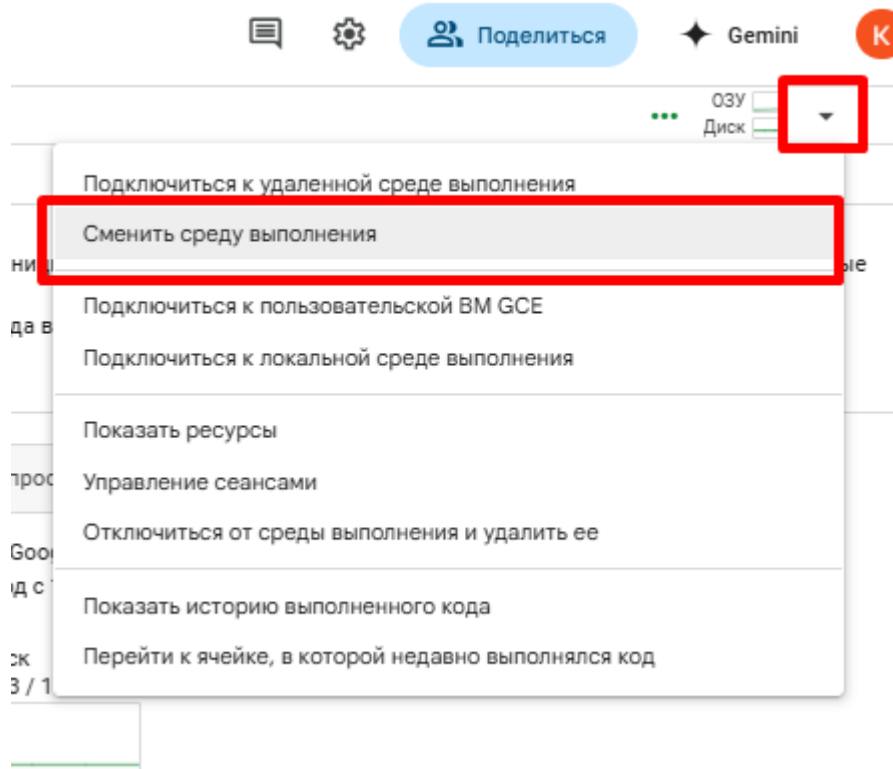
- обучение меньшего количества параметров, 10K-100K, потому что мы берём стандартную размерность эмбединга и умножаем на длину промпта;
- хорошие результаты для больших моделей, у которых больше 5B параметров;
- возможность адаптировать одну модель для выполнения различных задач.

6. Finetuning



💡 Перед запуском кода выбери правильную среду выполнения:

1. Нажми на стрелочку в правом верхнем углу.
2. Выбери пункт **Сменить среду выполнения**.



3. В поле **Аппаратный ускоритель** выбери **Графический процессор T4** (если доступно).

Сменить среду выполнения

Тип среды выполнения

Python 3

Аппаратный ускоритель ?

☒ ЦП ☒ Графический процессор T4

☐ Графический процессор A100

☐ Графический процессор L4 ☐ TPU v2-8

☐ TPU v5e-1

Нужен доступ к мощным графическим процессорам?
[Купите дополнительные вычислительные единицы](#)

Отмена **Сохранить**

4. Нажми **Сохранить**.

В этой практике мы рассмотрим, как можно файн-тюнить большие языковые модели без использования сложных техник, таких как PFT, LoRA или soft-prompts. В документации приведены хорошие примеры. В этом уроке мы разберем базовые вещи: подготовка данных, локализация и использование кастомных метрик при обучении.

1. Импортируем необходимые библиотеки:

```
!pip install transformers==4.35.2 datasets==2.16.1 evaluate==0.4.0 rouge-s  
!apt install git-lfs
```

2. Логинимся в Hugging Face

```
from huggingface_hub import notebook_login  
  
notebook_login()
```

▼ Напоминание: как получить токен для Hugging Face

Эта часть кода запросит у тебя токен Hugging Face. Как его получить:

```
from huggingface_hub import notebook_login

notebook_login()
```



Copy a token from [your Hugging Face tokens page](#) and paste it below.

Immediately click login after copying your token or it might be stored in plain text in this notebook file.

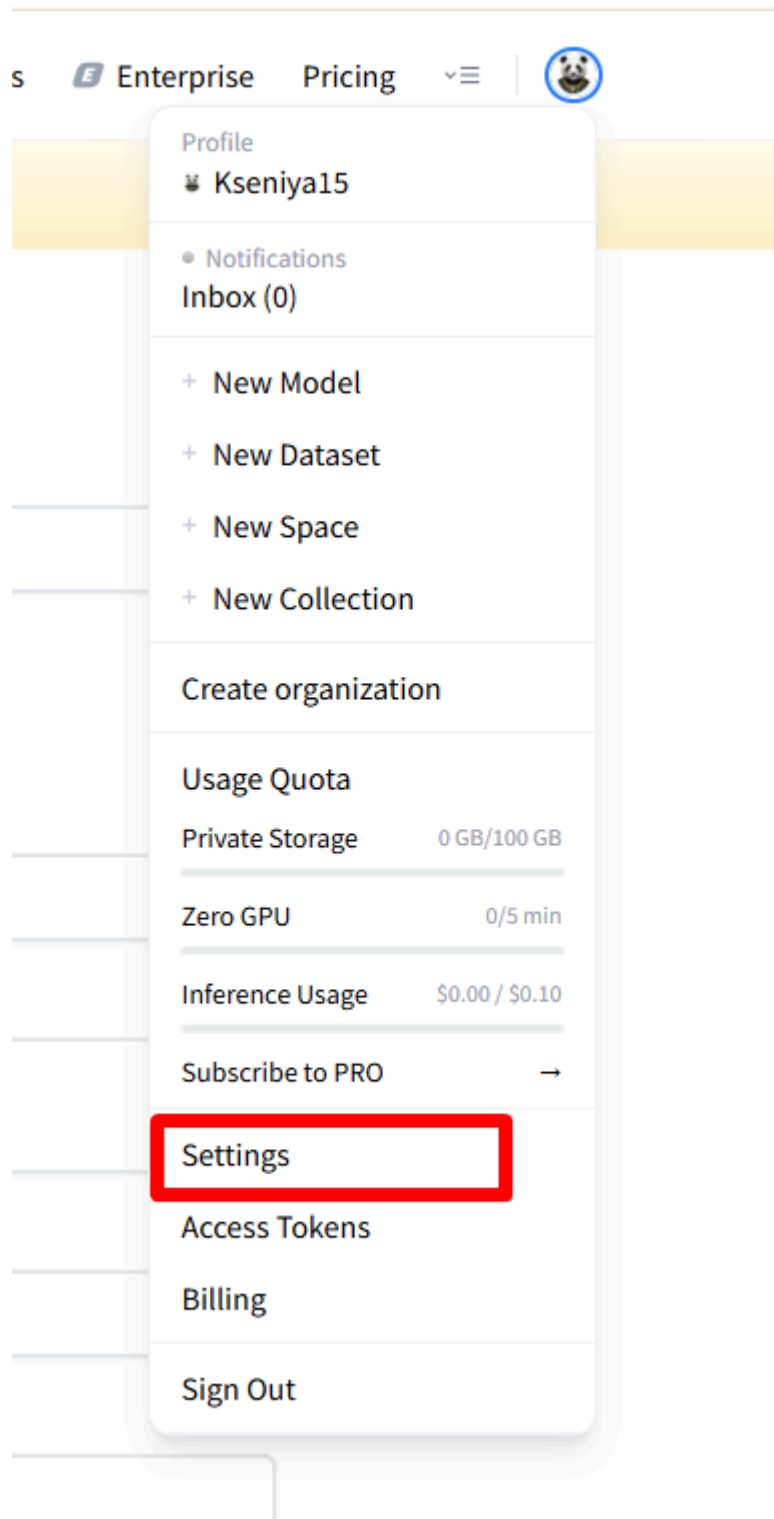
Token:

☒ Add token as git credential?

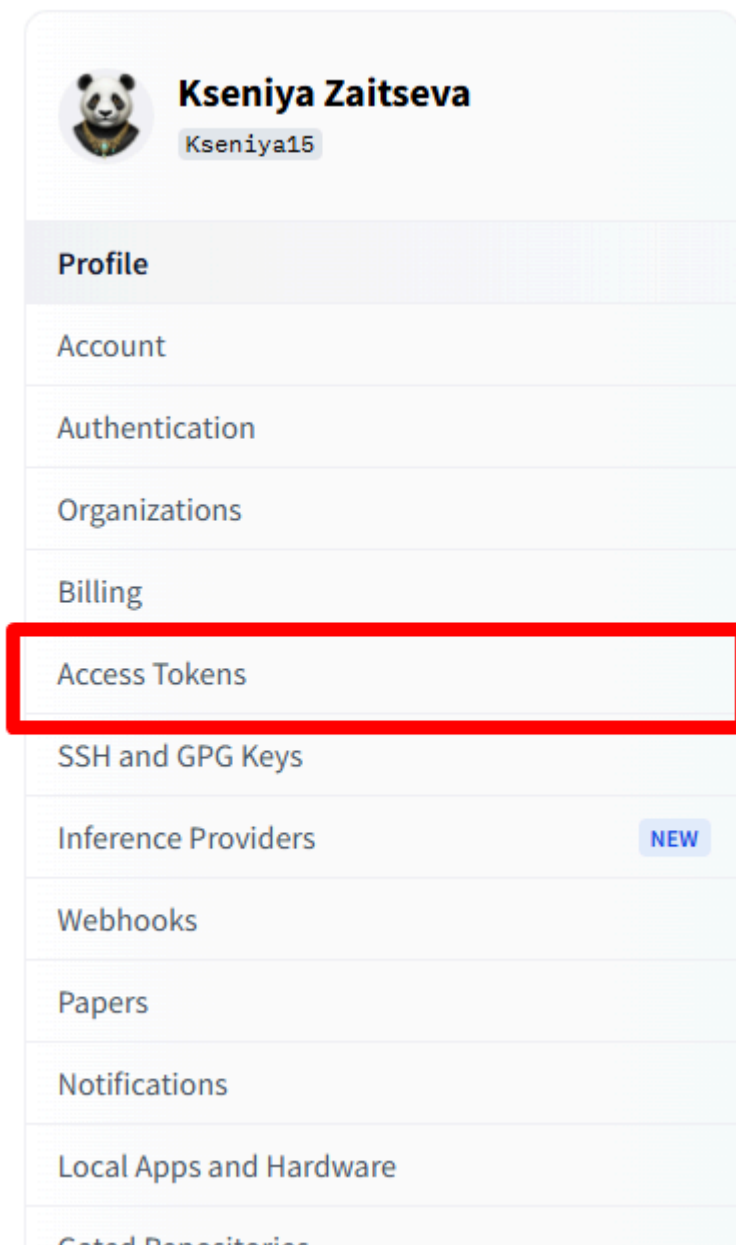
Login

Pro Tip: If you don't already have one, you can create a dedicated 'notebooks' token with 'write' access, that you can then easily reuse for all notebooks.

1. Переходим на сайт <https://huggingface.co/> (<https://huggingface.co/>) и **регистраемся** (если аккаунта ещё нет).
2. Нажимаем на аватарку в правом верхнем углу и выбираем **Settings**.



3. Кликаем по вкладке **Access Tokens**.



4. Вводим пароль (если запросят).

5. Жмём на кнопку **Create new token**.

Access Tokens

User Access Tokens

[+ Create new token](#)

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions.

⚠ Do not share your **Access Tokens** with anyone; we regularly check for leaked Access Tokens and remove them immediately.

Name ▾	Value	Last Refreshed Date ▾	Last Used Date ▾	Permissions ▾	
44443	hf_...rSLL	Jan 24	Jan 24	READ	⋮
3	hf_...QZEc	Jan 24	Jan 24	READ	⋮
444	hf_...gTqv	Jan 24	-	READ	⋮

6. Выбираем тип доступа — **Write**.

< Create new Access Token

Token type

Fine-grained

Read

Write

ⓘ This cannot be changed after token creation.

Token name

FTZero

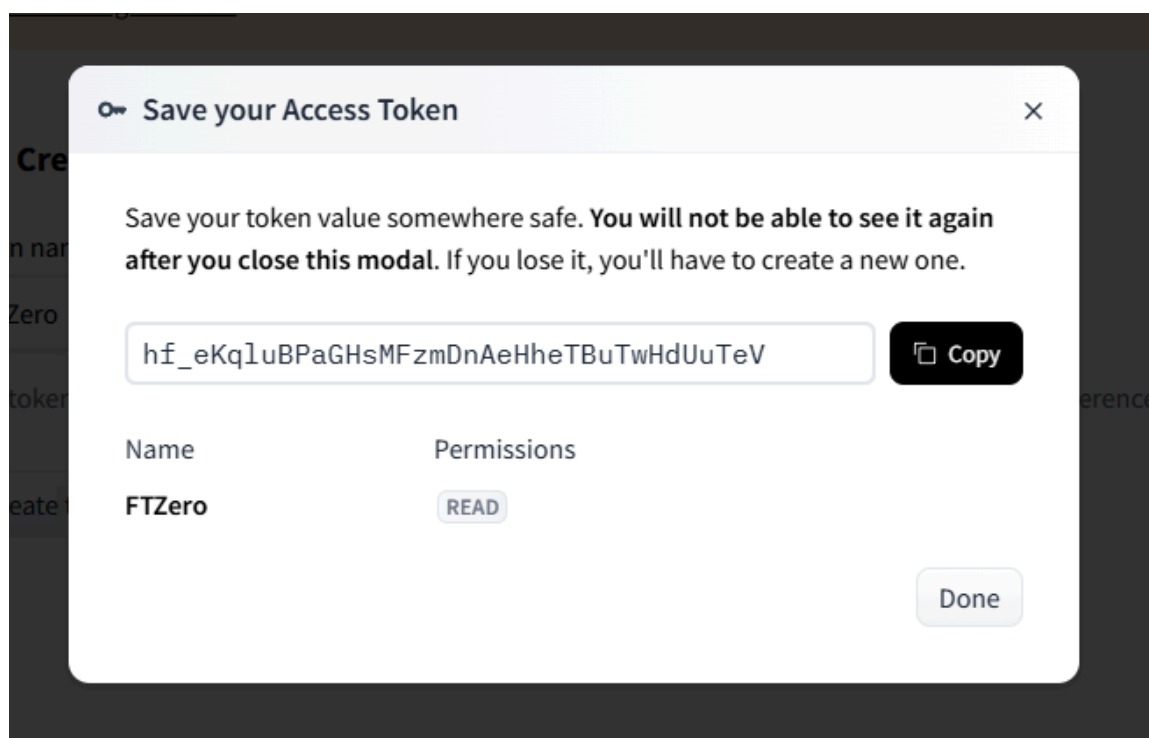
This token has read and write access to all your and your orgs resources and can make c

Create token

7. Придумываем имя токена (например, FTZero).

8. Жмём **Create token**.

9. Копируем токен и сохраняем его в надёжное место — он пригодится при запуске кода.



3. Импортирует библиотеку transformers

```
import transformers

print(transformers.__version__)
```

► Библиотека transformers

4. Задаём задачу. Мы будем дообучать модель **T5Small** на датасете **XSum**, который представляет собой экстремальную суммаризацию новостных статей с сайта BBC. Суммаризация здесь сводится к одному предложению, что делает задачу весьма сложной. Задаем название для чекпоинта модели. Уникальные названия моделей можно найти на сайте Hugging Face во вкладке "Models (<https://huggingface.co/models>)".

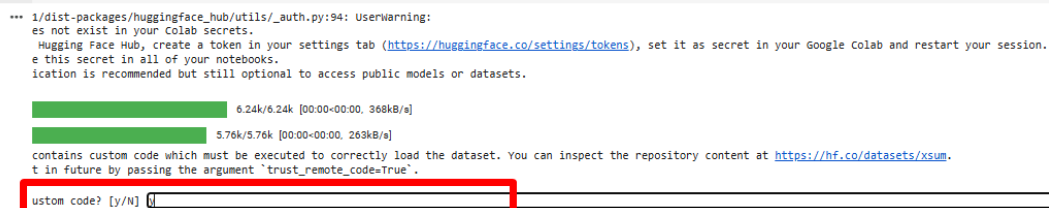
```
model_checkpoint = "t5-small" # выбираем модель
```

5. Сначала загрузим датасет и метрику из библиотек `datasets` и `evaluate`. Посмотрим на структуру датасета. Он содержит ключи `train`, `validation`, `test`, и каждую часть датасета представляет определенное количество примеров. Например, обучающий датасет содержит 204 тысячи примеров, каждый из которых имеет уникальный идентификатор, текст статьи и ее краткое `summary`.

```
from datasets import load_dataset
from evaluate import load

raw_datasets = load_dataset("xsum")
metric = load("rouge")
```

Если в коде появится вопрос — введи `y`.



```
... 1/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
  secrets not exist in your Colab secrets.
  Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
  Add this secret in all of your notebooks.
  Authentication is recommended but still optional to access public models or datasets.

6.24k/6.24k [00:00<00:00, 368kB/s]
5.76k/5.76k [00:00<00:00, 263kB/s]
contains custom code which must be executed to correctly load the dataset. You can inspect the repository content at https://hf.co/datasets/xsum.
to run in future by passing the argument 'trust_remote_code=True'.
custom code? [y/N] y
```

6. Выводим первый пример из обучающей выборки.

```
raw_datasets["train"][0]
```


7. Импортируем библиотеки и модули

```
import datasets
import random
import pandas as pd
from IPython.display import display, HTML
```

► Описание библиотек и модулей

8. Для работы с большими языковыми моделями необходимо токенизировать тексты, разбивая их на токены (слова, части слов, знаки препинания и т.д.), и присваивать каждому токenu уникальный идентификатор. Важно использовать тот же токенизатор, что и у изначальной модели.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

9. Проверяем токенизацию.

```
tokenizer("Hello, this one sentence!")
```

10. Прописываем функцию для токенизации всех примеров датасета. Для модели T5Small необходимо добавлять префикс к summary, чтобы модель понимала, что ей нужно выполнять суммаризацию. Отграничиваем длину запроса и выдачи.

```

max_input_length = 1024
max_target_length = 128

prefix = "summarize: "

def preprocess_function(examples):
    inputs = [prefix + doc for doc in examples["document"]]
    model_inputs = tokenizer(inputs, max_length=max_input_length, truncati

    # Setup the tokenizer for targets
    labels = tokenizer(text_target=examples["summary"], max_length=max_tar

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

```

11. Смотрим пример предобработанных данных.

```
preprocess_function(raw_datasets['train'][:2])
```

12. Уменьшаем размер датасета.

💡 В Google Colab существуют ограничения на бесплатные вычислительные ресурсы, которые могут меняться со временем. Поэтому большой датасет, используемый экспертом, скорее всего не удастся полностью обработать в рамках Colab. Чтобы ускорить обучение и избежать сбоев, мы уменьшаем размер датасета.

```

raw_datasets["train"] = raw_datasets["train"].shuffle(seed=42).select(range(5000))
raw_datasets["validation"] = raw_datasets["validation"].shuffle(seed=42).select(range(1000))

```

Таким образом, для дообучения будет использоваться только первые 5000 строк тренировочного набора и 1000 строк валидационного.

13. Токенизируем все датасеты, мапим их на новые версии.

```
tokenized_datasets = raw_datasets.map(preprocess_function, batched=True)
```

14. Инициализируем модель с чекпойнтом.

```
from transformers import AutoModelForSeq2SeqLM, DataCollatorForSeq2Seq, Seq2SeqTrainingArguments

model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

15. Задаём гипер-параметры: количество примеров для одновременного скормливания, имя модели как название папки, оценка качества по прохождению эпохи, learning rate. `push_to_hub=True` означает, что результат станет доступен на Hugging Face.

```
batch_size = 16
model_name = model_checkpoint.split("/")[-1]
args = Seq2SeqTrainingArguments(
    output_dir=f"{model_name}-finetuned-xsum",
    eval_strategy="epoch", # <--- вот оно!
    learning_rate=2e-5,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=1,
    predict_with_generate=True,
    fp16=True,
    push_to_hub=True,
)
```

16. Задаём “упаковщик”, который возьмёт датасет, объединит в матчи и скормит модели.

```
data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)
```

17. Используем метрику Rouge, а так же будем замерять длину сгенерированных саммарии.

```

import nltk
import numpy as np

nltk.download('punkt')

nltk.data.path.append('/root/nltk_data')

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)
    # Replace -100 in the labels as we can't decode them.
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    # Rouge expects a newline after each sentence
    decoded_preds = ["\n".join(nltk.sent_tokenize(pred.strip())) for pred in decoded_preds]
    decoded_labels = ["\n".join(nltk.sent_tokenize(label.strip())) for label in decoded_labels]

    # Note that other metrics may not have a `use_aggregator` parameter
    # and thus will return a list, computing a metric for each sentence.
    result = metric.compute(predictions=decoded_preds, references=decoded_labels)
    # Extract a few results
    result = {key: value * 100 for key, value in result.items()}

    # Add mean generated length
    prediction_lens = [np.count_nonzero(pred != tokenizer.pad_token_id) for pred in decoded_preds]
    result["gen_len"] = np.mean(prediction_lens)

    return {k: round(v, 4) for k, v in result.items()}

```

► Описание библиотек

18. Передаём гипер-параметры в Trainer — это метод, запускающий обучение модели.

```
trainer = Seq2SeqTrainer(  
    model=model,  
    args=args,  
    train_dataset=tokenized_datasets["train"],  
    eval_dataset=tokenized_datasets["validation"],  
    data_collator=data_collator,  
    compute_metrics=compute_metrics  
)
```

19. Запускаем обучение.

```
trainer.train()
```

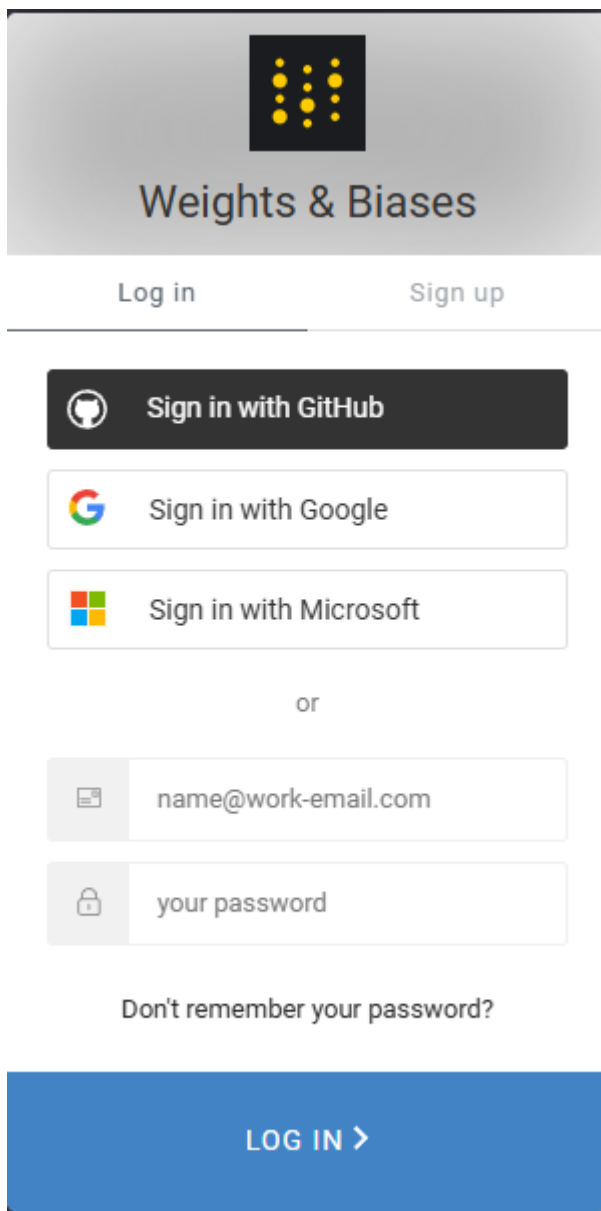
На этом этапе может появиться запрос на регистрацию в **Weights & Biases (wandb)** для получения API-ключа.



wandb (Weights & Biases) — это **инструмент для отслеживания экспериментов** в машинном обучении.

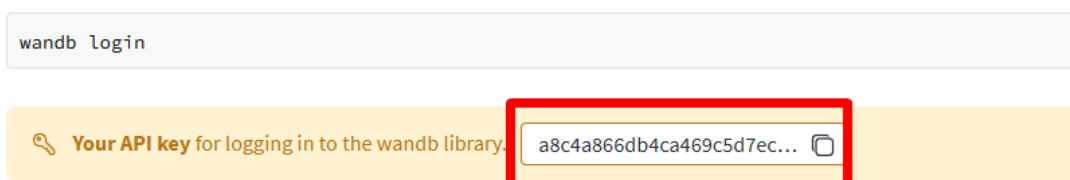
▼ Как получить API-ключ

1. Переходим на сайт авторизации: <https://wandb.ai/authorize>
(<https://wandb.ai/authorize>)
2. Регистрируемся с помощью Google-почты или любым другим удобным способом.



The image shows the Weights & Biases login page. At the top is the W&B logo and the text 'Weights & Biases'. Below this are 'Log in' and 'Sign up' links. There are three social login buttons: 'Sign in with GitHub', 'Sign in with Google', and 'Sign in with Microsoft'. Below these is an 'or' separator. Then there are two input fields: one for an email address (pre-filled with 'name@work-email.com') and one for a password (pre-filled with 'your password'). Below the password field is a link 'Don't remember your password?'. At the bottom is a large blue 'LOG IN >' button.

3. Заполняем информацию о себе → нажимаем **Continue**, затем ещё раз **Continue**.
4. Выбираем опцию **Models** и нажимаем **Continue**.
5. Копируем сгенерированный API-ключ.



2. Log a Run to a new project

6. Вставляем его в окно кода, когда система запросит, и нажимаем **Enter**.



20. По результатам обучения можно отслеживать метрики, такие как `training loss`, `validation loss`, значения ROUGE и среднюю длину предсказанных summary. Также отображается, сколько времени заняло обучение и сколько памяти GPU использовалось.

💡 Ты можешь отправить результаты дообучения в ChatGPT и попросить его подробно проанализировать их.

Таким образом, используя код из этого урока, можно фан-тюнить большие языковые модели с платформы Hugging Face на кастомных датасетах.

[12753/12753 1:21:48, Epoch 1/1]									
Epoch	Training Loss	Validation Loss	Rouge1	Rouge2	RougeL	RougeLsum	Gen Len	Runtime	Samples Per Second
1	2.721100	2.479327	28.300900	7.721100	22.243000	22.249600	18.822500	326.333800	34.725000
TrainOutput(global_step=12753, training_loss=2.7692033505520146, metrics={'train_runtime': 4909.3835, 'train_samples_per_second': 2.598, 'total_flos': 7.774481450954342e+16, 'epoch': 1.0, 'init_mem_cpu_alloc_delta': 335248, 'init_mem_gpu_alloc_delta': 242026496, 'init_mem_cpu_peaked_delta': 18306, 'init_mem_gpu_peaked_delta': 0, 'train_mem_cpu_alloc_delta': 2637782, 'train_mem_gpu_alloc_delta': 728138240, 'train_mem_cpu_peaked_delta': 138226182, 'train_mem_gpu_peaked_delta': 14677017088})									

Почему дообучение модели длится так долго?

- Используется **CPU**, а не GPU.
- **Медленная видеокарта** (например, T4 в Colab — не самая мощная).
- **Слишком много данных** для обучения.
- **Слишком много эпох** (обучение повторяется много раз).
- **Большой размер пакета данных (batch size)** — обрабатывается дольше.

► Инструменты и оборудование для дообучения

Дополнительный материал

🔗 Проект эксперта:

https://colab.research.google.com/drive/1dDd7YZb7KHvf2eZhR_ygutS_fNJwRlql?usp=sharing

(https://colab.research.google.com/drive/1dDd7YZb7KHvf2eZhR_ygutS_fNJwRlql?usp=sharing)

⚙ Проект эксперта с

обновлениями HuggingFace <https://colab.research.google.com/drive/1d-LiX37S3xmXoWs4EDXOijJAREBpMnCA?usp=sharing>

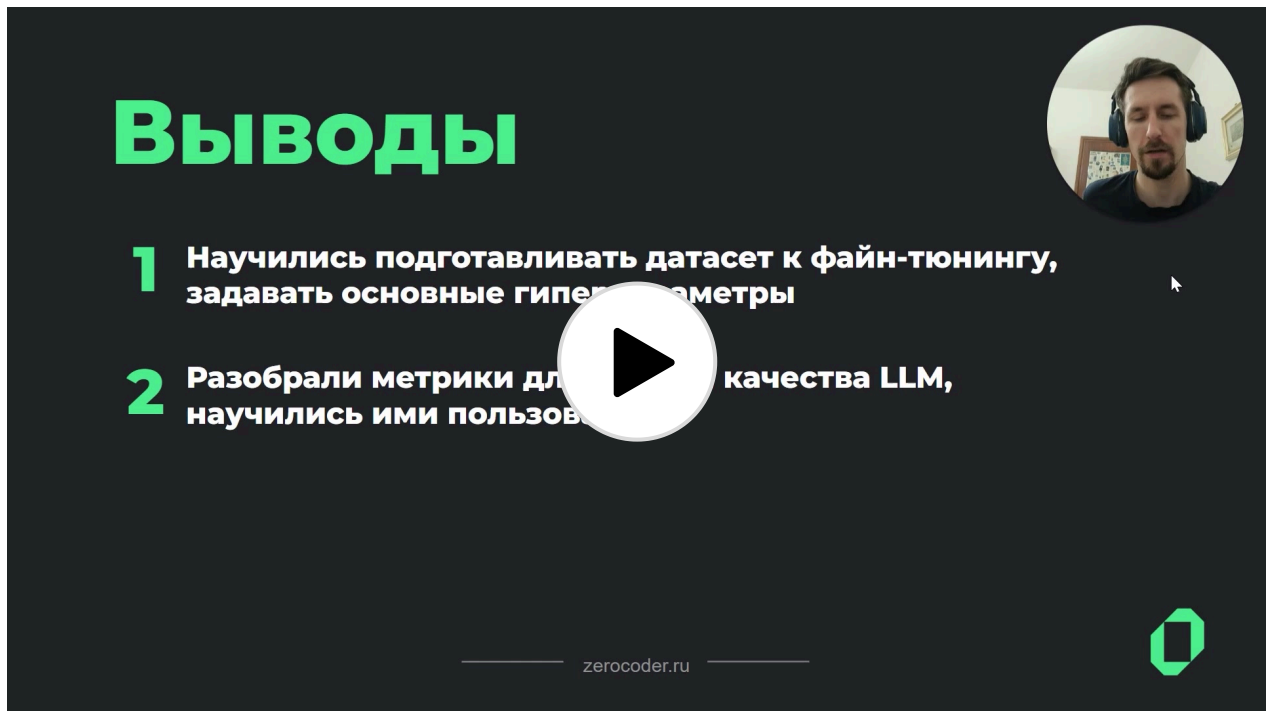
(<https://colab.research.google.com/drive/1d-LiX37S3xmXoWs4EDXOijJAREBpMnCA?usp=sharing>)

⚙ Презентация эксперта:

https://docs.google.com/presentation/d/1A5KPJd7cbITCW9l5x_cM_BD_HZzKLugn1cRSnO8Ycvic/edit?usp=sharing

(https://docs.google.com/presentation/d/1A5KPJd7cbITCW9l5x_cM_BD_HZzKLugn1cRSnO8Ycvic/edit?usp=sharing)

Основные выводы урока



Выводы

- 1 Научились подготавливать датасет к файн-тюнингу, задавать основные гиперпараметры
- 2 Разобрали метрики для оценки качества LLM, научились ими пользоваться

zerocoder.ru

- научились подготавливать датасет к файн-тюнингу, задавать основные гиперпараметры;
- разобрали метрики для оценки качества LLM, научились ими пользоваться;
- обсудили, как работает PEFT.

Домашнее задание

Самостоятельно реализуй fine-tuning модели T5Small на датасете, который использовал эксперт в своём проекте (или используй собственный датасет):

1. Установи библиотеки и инструменты.
2. Определи модель T5Small и датасет XSum .
3. Загрузи датасет (команда, чтобы его загрузить: `load_dataset("xsum")`), загрузи метрику ROUGE , посмотри на структуру датасета.
4. Реализуй токенизацию данных.
5. Инициализируй модель T5Small .
6. Задай гипер-параметры для обучения модели.
7. Создай "упаковщик" данных для обучения модели.
8. Напиши функцию для вычисления метрик ROUGE .
9. Передай все параметры в Seq2SeqTrainer .
10. Запусти процесс обучения модели.
11. Отслежи метрики обучения, такие как `training loss` , `validation loss` , ROUGE , среднюю длину предсказанных summary, время обучения и использование GPU.

Представь домашнее задание в виде скриншотов, иллюстрирующих выполнение каждого этапа.

На следующем занятии

На следующем уроке мы погрузимся в тему создания поисковика на базе FAISS.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

Самостоятельно реализуй fine-tuning модели `T5Small` на датасете, который использовал эксперт в своём проекте (или используй собственный датасет):

1. Установи библиотеки и инструменты.
2. Определи модель `T5Small` и датасет `XSum`.
3. Загрузи датасет (команда, чтобы его загрузить: `load_dataset("xsum")`), загрузи метрику `ROUGE`, посмотри на структуру датасета.
4. Реализуй токенизацию данных.
5. Инициализируй модель `T5Small`.
6. Задай гипер-параметры для обучения модели.
7. Создай "упаковщик" данных для обучения модели.
8. Напиши функцию для вычисления метрик `ROUGE`.
9. Передай все параметры в `Seq2SeqTrainer`.

10. Запусти процесс обучения модели.

11. Отследи метрики обучения, такие как `training loss` , `validation loss` , ROUGE , среднюю длину предсказанных summary, время обучения и использование GPU.

Представь домашнее задание в виде скриншотов, иллюстрирующих выполнение каждого этапа.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Следующий урок